

A Verification Framework for State-Dependent MRDTs: describe only the sensible operations

Sal metatheory note — framework draft
FP Launchpad, IIT Madras

July 4, 2026

Abstract

Mergeable replicated data types (MRDTs) are verified by a standard division of labour: the data-type author discharges a fixed catalogue of local, equational *verification conditions* (VCs), and a once-and-for-all soundness meta-theorem lifts them to RA-linearizability of every reachable configuration. Every published catalogue quantifies its VCs *over all states*. This is adequate for *flat* data types — sets, counters, registers — whose operations are total: meaningful on every state. It is *inadequate* for *state-dependent* data types, whose operations are only meaningful on well-formed states and whose effect depends on the state they observe. The sharp cases are tombstone-free sequence and rich-text CRDTs (a physical-delete RGA; Peritext without tombstones): deletion physically removes a node and *re-anchors* its dependents, so an operation’s recorded position can become stale, and its commutation with a concurrent operation holds only on reachable states. Whole-state VCs are simply *false* on the infeasible states such a type admits.

We present a *conditioned* verification framework. The author enriches the signature with two predicates — an *applicability* predicate `app` (“when is this operation sensible?”) and a reachability invariant `lnv` — and discharges VCs *conditioned* on them; the framework proves RA-linearizability once and for all. The developer thus *describes only the sensible operations* (as the RGA already does with insertion anchors) and is never asked to reason about infeasible states. The technical core is an *applicability-aware* linearization order together with a commutation discipline — path-faithfulness plus causal freshness — under which state-dependent, re-anchoring operations provably commute. The development is mechanized in Lean 4; we state which results are kernel-checked and which are in progress.

1 Introduction

An MRDT is a replicated data type whose replicas fork, apply operations locally, and reconcile divergence by a three-way *merge* `merge(l, a, b)` relative to the lowest common ancestor l of the two divergent states a, b . Correctness is *RA-linearizability*: every version in the store is explained by a sequencing of exactly the events it has observed, consistent with a linearization order lo derived from visibility and a conflict-resolution relation rc .

The prevailing methodology reduces this global property to a fixed catalogue of *local, equational* VCs over `do`, `merge` and `rc`, discharged per data type, and a soundness meta-theorem, proved once, that lifts the VCs to RA-linearizability of every reachable configuration. Crucially, the VCs are stated as universally quantified equations *over all states* $\sigma \in \Sigma$.

The whole-state assumption, and where it breaks. Universal quantification is benign when every operation is *total* — meaningful on every state — and *state-independent* in the sense that a VC true on the reachable states is true on all states. Flat data types are of this kind: a grow-only set’s `add` inserts on any state; an OR-set’s `rem` deletes whatever tags it sees. But consider a *tombstone-free* sequence CRDT (an RGA that physically deletes):

- An insertion records the *position* it attaches to (a path to an anchor node). If a concurrent deletion has removed that anchor, the insertion must *climb* to the nearest surviving ancestor — its effect *depends on the state*.
- A deletion physically removes a node and *re-anchors* (rehomes) its children to the deleted node’s parent.
- Consequently two operations commute only on *well-formed* states: replayed on a syntactically-constructible but unreachable state, the equation is false.

A whole-state VC catalogue cannot express such a type: its commutation VC is false as a $\forall\sigma$ statement, even though it holds on every state the system can actually reach. This is not a peculiarity of the RGA; it is the generic situation for any data type with *structural* state and *positional* operations. Peritext without tombstones is the same phenomenon one level up: a formatting mark anchors to character positions, and deleting a character re-anchors the mark.

Thesis. Whole-state VC catalogues *over-specify*: they demand equations on states no execution reaches. The remedy is not to invent new equations but to *condition* the existing ones on what is reachable and sensible, and to re-prove the meta-theorem to consume the conditioned forms. The author’s added burden should be exactly the *description of sensible operations* — which, for the RGA, is already implicit in the anchor discipline — and nothing more.

Contributions.

1. A *conditioned signature* (§3) carrying an applicability predicate `app` and a reachability invariant `Inv`, with a *collapse* property: flat types take `app = Inv =  $\top$` and recover the ordinary framework verbatim.
2. A *conditioned VC discipline* (§5): commutation and the linearization order are relativized to `app` and `Inv`, so the author discharges VCs *assuming sensibility*.
3. The technical core (§6): an *applicability-aware* linearization order and a commutation discipline — *path-faithfulness* and *causal freshness* — under which state-dependent, re-anchoring operations commute; and the convergence theorem this yields.
4. A once-and-for-all *adequacy* theorem (§7) reducing RA-linearizability to the conditioned VCs, with an explicit division between the author’s obligations and the framework’s generic proof.
5. Instantiation (§8): the tombstone-free RGA as the driving instance, and Peritext-without-tombstones as an instance of the same *anchor-carrying* class that reuses the commutation discipline.

2 The replicated model and RA-linearizability

We recall the model, briefly, to fix notation; the framework of this note is orthogonal to its details.

An *event* is a triple $e = (t, r, o)$: a globally unique timestamp t , an origin replica r , and an operation o . A *configuration* carries a *store*: a directed acyclic graph of *versions*, each holding a state $\sigma \in \Sigma$ and the set $L(v)$ of events that produced it, with per-replica heads. A transition forks a replica, applies a fresh event ($\sigma \mapsto \text{do}(\sigma, e)$; we write $e(\sigma)$), queries, or **merges**: it picks two heads v_1, v_2 , finds their lowest common ancestor v_ℓ , and installs a new version with state $\text{merge}(\sigma_{v_\ell}, \sigma_{v_1}, \sigma_{v_2})$ and event set $L(v_1) \cup L(v_2)$. Visibility $\xrightarrow{\text{vis}}$ records what each event's issuing replica had seen; $e_1 \parallel e_2$ denotes concurrency ($\neg(e_1 \xrightarrow{\text{vis}} e_2)$ and $\neg(e_2 \xrightarrow{\text{vis}} e_1)$). Two events *commute*, $e_1 \rightleftharpoons e_2$, when $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$ for all σ .

Definition 2.1 (RA-linearizability, per version). *A list $\pi = e^1 \dots e^n$ witnesses a version holding state s with event set E if (i) π enumerates E without repetition; (ii) π extends the linearization order lo ; and (iii) it folds to the state, $e^n(\dots e^1(\sigma_0) \dots) = s$. A configuration is RA-linearizable if every version in its store admits a witness.*

The order lo places e_1 before e_2 when either $e_1 \xrightarrow{\text{vis}} e_2$ and $e_1 \not\rightleftharpoons e_2$, or $e_1 \parallel e_2$, $e_1 \not\rightleftharpoons e_2$, rc orders them, and e_2 is not already overwritten by a later non-commuting event. We work throughout with the *set-relative* refinement lo^E of this order, in which the overwriter clause ranges over the event set E of the version being linearized rather than the whole configuration; this refinement is forced (an event can gain overwriters from a concurrent branch) and $\text{lo} \subseteq \text{lo}^E$ pointwise, so every lo^E -witness is an lo -witness.

3 The conditioned signature

Definition 3.1 (conditioned MRDT signature). *A conditioned MRDT signature is a tuple*

$$\mathcal{D} = \langle \Sigma, \sigma_0, \text{do}, \text{merge}, \text{rc}, \approx, \text{Inv}, \text{app} \rangle$$

extending an ordinary MRDT signature $\langle \Sigma, \sigma_0, \text{do}, \text{merge}, \text{rc} \rangle$ with

- *an observational equivalence $\approx \subseteq \Sigma \times \Sigma$ — an equivalence relation that is a congruence for do and merge (Def. 3.2); on flat types it may be taken as structural equality;*
- *a reachability invariant $\text{Inv} : \Sigma \rightarrow \text{Prop}$ — an over-approximation of the reachable states;*
- *an applicability predicate $\text{app} : \text{Op} \times \Sigma \rightarrow \text{Prop}$ — “operation o is sensible on state σ .”*

A signature is flat when $\text{Inv} = \top$, $\text{app} = \top$, and $\approx$ is structural equality; then all conditioned notions below reduce definitionally to the ordinary ones.

Definition 3.2 (observational equivalence). $\sigma_1 \approx \sigma_2$ *iff every query supported by the data type returns the same value on σ_1 and σ_2 . Convergence is stated up to $\approx$: two replicas that have observed the same events must be observationally equivalent, not necessarily structurally equal.*¹

¹This is the notion of *convergence modulo observable behaviour*: two states may differ in representation (a balanced vs. unbalanced search tree; a physically-deleted node left as unreachable junk) yet agree on every observation. For the tombstone-free RGA it turns out that reachable convergence holds even *structurally*, so $\approx$ may be taken as equality; we keep it abstract because state-dependent types in general need it.

Running example: the tombstone-free RGA. The state is a forest over node identifiers: Σ maps each live identifier t to a payload and an anchor $\text{anc}(t)$ (its parent), with a root sentinel 0. We write $\text{live}(\sigma, t)$ for “ t is present in σ ”. Two operations:

$$\begin{aligned} \text{Ins}(e, p, a) : \quad & \text{do}(\sigma, \text{Ins}(e, p, a)) = \sigma[t \mapsto (e, \text{resolve}(\sigma, a:p))], \\ \text{Del}(p, x) : \quad & \text{rehome each child of } x \text{ to } \text{resolve}(\sigma, p), \text{ then remove } x, \end{aligned}$$

where $\text{resolve}(\sigma, L)$ returns the first identifier of the list L that is live in σ , or the root 0 if none is (the *climb*). An insertion records a *path* p and an *anchor* a ; a deletion records the path p to which its children rehome. The applicability and invariant predicates capture “sensible”:

$$\begin{aligned} \text{app}(\text{Ins}(e, p, a), \sigma) &\equiv (a = 0 \wedge p = []) \vee (\text{live}(\sigma, a) \wedge p \text{ is the true ancestor chain of } a), \\ \text{app}(\text{Del}(p, x), \sigma) &\equiv \text{analogously, } x \text{ live and } p \text{ its true rehome path,} \\ \text{Inv}(\sigma) &\equiv \text{wf}(\sigma) \wedge \text{idmono}(\sigma), \end{aligned}$$

where wf says the forest is well-formed (every anchor live or the root) and idmono says anchors strictly decrease ($\text{anc}(\sigma, t) = 0 \vee \text{anc}(\sigma, t) < t$), a consequence of monotone timestamp allocation. The anchor discipline the RGA already carries *is* its applicability predicate; the author writes nothing new to describe sensible operations.

Example 3.3 (the unconditioned commutation VC is false). *This is why conditioning is necessary, not merely convenient. Take two insertions issued concurrently by different replicas: a records anchor 3 and path [7] (it creates a node under 3, climbing the path if 3 is gone), and b creates a node with identifier 7 at the root. Evaluate the two orders on the initial root-only state σ_0 :*

- $\text{do}(\text{do}(\sigma_0, a), b)$ — *a first: it resolves its list [3, 7] against σ_0 , where both 3 and 7 are absent, and climbs to the root, anchoring its node at 0; then b creates node 7.*
- $\text{do}(\text{do}(\sigma_0, b), a)$ — *b first: it creates node 7; now a resolves [3, 7] with 3 absent but 7 live, anchoring its node at 7.*

The two results disagree on the parent of a’s node (0 versus 7): the whole-state commutation equation

$$\forall \sigma. \quad \text{do}(\text{do}(\sigma, a), b) = \text{do}(\text{do}(\sigma, b), a)$$

*is therefore false, already at $\sigma = \sigma_0$ (**naive_general_swap_false**). No execution reaches this counterexample: a records an anchor 3 that is not present and a path naming b’s identifier 7, yet a and b are concurrent — a’s replica never observed b, so it could neither anchor to an absent node nor name b’s fresh identifier. These are exactly the tuples **app** (and causal freshness, §6) excludes; restricted to applicable tuples, the two insertions do commute (Lemma 6.5). A whole-state catalogue quantifies its commutation VC over $\sigma = \sigma_0$ with this a and so cannot state a true VC for the type at all; the conditioned catalogue can. The RGA already encodes the fix — its shipped commutation VC is gated on $\text{accurate} \wedge \text{fresh} \wedge \text{wf}$ precisely because the ungated equation is this counterexample.*

4 Sensible operations and reachability

The two predicates play complementary roles, and each carries exactly one obligation.

Definition 4.1 (reachability invariant). *Inv is a reachability invariant if $\text{Inv}(\sigma_0)$ holds and Inv is preserved by every transition: for all σ with $\text{Inv}(\sigma)$ and every applicable event e , $\text{Inv}(e(\sigma))$; and for all Inv-states l, a, b arising as an honest LCA-and-branches triple, $\text{Inv}(\text{merge}(l, a, b))$. We write Reach for the states satisfying Inv.*

Establishing Inv as a reachability invariant is the author’s obligation; it is a routine induction over do and merge (for the RGA, preservation of wf under merge requires exactly idmono, which the same induction supplies). No other whole-state reasoning is asked of the author.

Remark 4.2 (the developer-effort ledger). *The essential/author column and the generic/framework column are:*

<i>Author provides (essential)</i>	<i>Framework proves (once)</i>
app — spec of sensible operations	convergence of concurrent operations
Inv + reachability-invariance (Def. 4.1)	existence of canonical states $\sigma(E)$
conditioned VCs (§5, assuming app)	the linearization machinery
	RA-linearizability (Thm. 7.2)

The author’s column is the specification — one cannot verify a state-dependent type without saying which states are valid and which operations sensible — and no more. The failure mode of a whole-state framework is that it leaks framework-column work into the author’s lap; §6 shows how the conditioned framework keeps the columns separate.

5 The conditioned verification conditions

The single primitive is conditioned commutation.

Definition 5.1 (conditioned commutation). *Operations o_1, o_2 commute where sensible, written $o_1 \rightleftharpoons_{\text{app}} o_2$, if for every σ with $\text{Inv}(\sigma)$, $\text{app}(o_1, \sigma)$ and $\text{app}(o_2, \sigma)$,*

$$\text{do}(\text{do}(\sigma, o_1), o_2) \approx \text{do}(\text{do}(\sigma, o_2), o_1).$$

Commutation is demanded only at reachable states where both operations are sensible, and only up to observation. The relational VCs on rc are the usual ones with $\rightleftharpoons$ replaced by $\rightleftharpoons_{\text{app}}$: RC-NON-COMM (non-commuting distinct-replica operations are rc-ordered in exactly one direction), NO-RC-CHAIN (no $o_1 \xrightarrow{\text{rc}} o_2 \xrightarrow{\text{rc}} o_3$), and a conditioned COND-COMM (a resolved conflict is invisible once an overwriter runs). These are the author’s update-layer obligations, discharged *assuming* applicability of the operations involved — which, for state-dependent types, is what makes them provable at all.

6 Canonical states and convergence

This section is the technical core: how conditioned VCs yield a well-defined canonical state per event set, and the commutation discipline that makes it work for re-anchoring operations.

6.1 The applicability-aware order

The set-relative order lo^E (§2) orders *non-commuting* pairs. For state-dependent operations a second source of ordering appears: one operation’s *applicability* may depend on another. A concurrent deletion can render an insertion’s recorded anchor stale; the insertion still commutes with the deletion (it climbs), but the two are related by a *generation dependency* that the order must record, lest a reordering apply an operation at a state where it is neither sensible nor a harmless no-op.

Definition 6.1 (generation dependency; applicability-aware order). *Event e_2 depends on e_1 , written $e_1 \prec_{\text{app}} e_2$, if e_1 can change whether e_2 is applicable: $\exists \sigma. \text{app}(e_2, \sigma) \neq \text{app}(e_2, e_1(\sigma))$. The applicability-aware order is*

$$\text{lo}_{\text{app}}^E = \text{lo}^E \cup \{(e_1, e_2) : e_1 \xrightarrow{\text{vis}} e_2 \wedge e_1 \prec_{\text{app}} e_2\}.$$

For the RGA, $\prec_{\text{app}}$ is the syntactic fact “ e_2 ’s recorded path names e_1 ’s identifier”; it is discharged directly from the operation shapes.

Definition 6.2 (no-op-feasible enumeration). *A list $\pi = e^1 \cdots e^n$ is no-op-feasible if at each prefix fold $\sigma_i = e^i(\cdots e^1(\sigma_0) \cdots)$ the next event is either applicable at σ_i or acts as the identity there ($e^{i+1}(\sigma_i) \approx \sigma_i$).*

Two feasibility repairs are needed and are *complementary*, not alternatives: lo_{app}^E excludes reorderings that violate a generation dependency (which mere no-op tolerance would wrongly admit), while no-op tolerance admits the harmless *redundant* reorderings that strict applicability would wrongly reject (e.g. two concurrent deletions of one node: whichever runs second is a no-op, so no fully-applicable enumeration exists, yet both orders converge). The order handles the first; no-op-feasibility the second.

6.2 The commutation discipline for anchor-carrying types

The convergence proof reorders an enumeration to a canonical form by adjacent swaps of lo_{app}^E -incomparable events. Each swap must hold *at the prefix fold where the pair sits* — a state at which either event may be stale, because prior events in the prefix may have deleted its anchor. The discipline below identifies exactly what makes such a swap valid without demanding either event be applicable there.

Definition 6.3 (path-faithfulness). *An operation is faithful at σ if its recorded list tracks the true tree of σ above its climb target: for an insertion with list L , writing $c = \text{resolve}(\sigma, L)$, if c is live then $\text{resolve}(\sigma, L \setminus \{c\}) = \text{anc}(\sigma, c)$ (the list continues up the real ancestor chain); a deletion is faithful if its rehome path likewise resolves to its target’s true parent. Write $\text{Faithful}(o, \sigma)$.*

Definition 6.4 (causal freshness). $\text{NoFreshClash}(a, b)$ holds if b ’s (fresh) identifier does not occur in a ’s recorded list — the condition a real execution guarantees between concurrent events, since neither observed the other and identifiers are allocated afresh.

Faithfulness is strictly weaker than applicability ($\text{app}(o, \sigma) \Rightarrow \text{Faithful}(o, \sigma)$) and, unlike applicability, is *stable under re-anchoring*: a deletion that rehomes nodes preserves the faithfulness of a concurrent operation’s recorded path, because rehoming updates the true tree consistently with the climb. This is the property that lets the swap fire at stale states.

Lemma 6.5 (faithful commutation). *Let σ satisfy `Inv` with root absent, and let a, b be distinct-timestamp events with `Faithful`(a, σ), `Faithful`(b, σ), `NoFreshClash`(a, b) and `NoFreshClash`(b, a). Then*

$$\text{do}(\text{do}(\sigma, a), b) \approx \text{do}(\text{do}(\sigma, b), a).$$

general_swap_bothFaithful

Lemma 6.5 is mechanized (kernel-checked, all four insertion/deletion combinations). It is the reusable heart of the framework for anchor-carrying types: it says two re-anchoring operations commute whenever each is merely faithful and they do not name each other’s fresh identifiers — neither need be applicable. The asymmetric special case (b applicable, a faithful) is `general_swap`; that the symmetric case also holds is what guarantees the convergence swap fires *at every* prefix fold, where both events may be stale.

6.3 Convergence

Faithfulness must hold not merely at one state but at every prefix fold the canonicalization visits. This is a reachability-style invariant — call it *faithful-threading* — and it is where the framework absorbs the work that a whole-state approach would push onto the author.

Proposition 6.6 (faithful-threading, target form). *Along any lo_{app}^E -respecting, no-op-feasible enumeration of a backward-closed E , every event is faithful at the prefix fold at which it is applied.*

Threading is established from the author’s `app/Inv` data by a single-step preservation argument: faithfulness is preserved by an applicable insertion, and by an applicable deletion (the deletion splices its target out of the true chain, re-linking children to the target’s successor). The insertion step and the correct invariant strengthening are mechanized; the deletion step is the one remaining lemma (§9).

Theorem 6.7 (conditioned convergence, target form). *Assume the conditioned VCs (§5), Lemma 6.5, and Proposition 6.6. Then any two enumerations of a backward-closed event set E that respect lo_{app}^E and are no-op-feasible fold from σ_0 to $\approx$ -equivalent states.*

Convergence licenses the *canonical state* $\sigma(E)$ — the $\approx$ -class obtained by folding any such enumeration — and reduces “a version linearizes” to “its stored state $\approx \sigma$ of its event set,” with witness lists eliminated from all downstream reasoning. The proof is the ordinary bottom-up canonicalization (proved once for flat types), with each adjacent swap discharged by Lemma 6.5 under Proposition 6.6; nothing in it mentions `resolve`, anchors, or the RGA. That is the sense in which convergence is generic: it consumes the faithful-commutation VC (Lemma 6.5) and the threading obligation (Proposition 6.6) as hypotheses, and each data type supplies those from its own operation algebra.

7 Adequacy

Convergence is the update-layer half. The merge-layer half is a single equation between canonical states.

Definition 7.1 (Join Lemma). *For backward-closed event sets E_1, E_2 of a reachable configuration, $\sigma(E_1 \cup E_2) \approx \text{merge}(\sigma(E_1 \cap E_2), \sigma(E_1), \sigma(E_2))$.*

By the LCA property $L(v_\ell) = L(v_1) \cap L(v_2)$, the first argument is exactly the state the store hands the merge. The two halves compose once and for all:

Theorem 7.2 (adequacy). *If $\mathcal{D}$ satisfies the conditioned VCs, has `lnv` as a reachability invariant, and satisfies the Join Lemma, then every configuration reachable in the transition system is RA-linearizable, per version.*

The adequacy bridge is closure-indexed and proved generically: because every store version is fully causally closed, the Join Lemma applies at it regardless of the closure strength a data type declares, and the two established verification routes (a delta contract for set- and counter-shaped merges; a direct join for counter-comparison merges) are its two instances. The bridge and its instances are mechanized (`ra_linearizable3_of_joinC`), with the nine flat production MRDTs of the repository routed through it.

8 The class map and instances

Data types occupy a three-axis space: the *delta class* of the merge (group, lattice, or feasibility-conditioned), the *closure strength* the Join Lemma needs, and the *state invariant* `lnv`. Flat types sit on the `lnv =  $\top$` plane and are handled by the ordinary framework (the collapse of Def. 3.1). The conditioned framework of this note is what populates the `lnv  $\neq \top$` plane.

Tombstone-free RGA. The driving instance: `app` = the anchor discipline, `lnv =  $\text{wf} \wedge \text{idmono}$` , faithful commutation (Lemma 6.5) discharged from `resolve/rehome` algebra.

Peritext without tombstones. The same class, one level up. A formatting mark anchors to character positions as an RGA node anchors to its parent; deleting a character re-anchors the marks that span it, exactly the rehomeing phenomenon. The author supplies `app` (a mark’s endpoints anchor live characters), `lnv` (well-formed marks over a well-formed sequence), and discharges the conditioned VCs by the *same* faithfulness/freshness discipline (Lemmas 6.5 and the threading of Prop. 6.6), with marks reusing the anchor machinery. It is thereby an *instance*, not a second bespoke development — which is the payoff of a framework over a per-type proof.

9 Mechanization status

The development is in Lean 4 (Mathlib; axioms `{propext, Classical.choice, Quot.sound}`; no `sorry` in the cited results). Honestly delineated:

- **Kernel-checked.** The motivating counterexample (Example 3.3, `naive_general_swap_false`); the closure-indexed adequacy bridge (`ra_linearizable3_of_joinC`) and the nine flat MRDTs routed through it; the set-relative convergence for flat types; the conditioned-convergence order reduction; the commutation discipline for the RGA — Lemma 6.5 (`general_swap_bothFaithful`) and its asymmetric form (`general_swap`), all insertion/deletion combinations; and *both preservation steps* of the faithful-threading invariant (feeding Prop. 6.6) — insertion (`chainFaithful_doIns`) and the deletion splice (`chainFaithful_doDel`), the latter the keystone, whose induction outputs faithfulness on the target-free list and bridges back, dissolving the level-mismatch the splice creates. It is independently corroborated by property-based testing (a checker proved equivalent to the invariant; $\sim 58,000$ random reachable-state trials, no counterexample).

- **In progress — with the central obstruction now resolved.** Theorem 6.7 itself. The subtlety was faithful-threading (Prop. 6.6): naively threading faithfulness for *all* pending events at *all* reordered folds is false (transiently, while a pending event’s ancestors are still being folded), which defeated three successive attempts. The resolution is to scope the invariant to *enabled* events — those whose causal past has been folded, the only ones the canonicalization ever swaps. On that scope threading holds, and its two steps are mechanized (kernel-clean): an incomparable step — a fresh concurrent insertion or a *faithful* deletion — preserves the invariant (`chainFaithful_incompStep`, which also upgrades the deletion step from *accurate* to merely *faithful*), and it threads along a reordering (`chainFaithful_incompFold`). Property-based testing on a two-replica concurrent generator (hundreds of scenarios $\times$ all interleavings) reports zero convergence failures. What remains is *plumbing*: an enablement base case, a visibility formalization, and the observational-equivalence ($\approx$) state layer feeding $\sigma(E)$ — no semantic obstruction is known to stand in the way, and the alternative (base-anchored / applicability-preserving) canonicalization is confirmed *not* required.
- **Target.** End-to-end hosting of the RGA (and then Peritext) as conditioned instances, by composing Theorem 6.7 with the adequacy bridge of §7. We have verified, on reachable states, that the RGA converges even structurally, so $\approx$ may there be taken as equality — a simplification, not a soundness requirement.

Relation to prior methodologies. Whole-state VC catalogues (Neem) are the $\text{Inv} = \text{app} = \top$ collapse of this framework. The use of observational equivalence rather than structural equality follows earlier work on *convergence modulo observable behaviour* (Peepul); the contribution here is to combine it with applicability conditioning and to isolate the commutation discipline (faithfulness + freshness) that makes state-dependent, re-anchoring operations tractable, so that the author need only *describe the sensible operations*.